Honey, I Tiled the Tensors

Shapes, Strides, Swizzles and Suffering! - An intro to Layout Algebra

Published: February 26, 2026 **Reading time:** 18 min read **Tags:** #ml #cuda #gpu

Layouts are a powerful abstraction introduced in NVIDIA's CuTe library for making operations on complicated Tensor configurations a little bit easier to understand. My goal here is to provide a good taste of how operations on these layouts work and an example matrix-matrix multiplication kernel to show the value of these abstractions and its drawbacks. This is not a full mathematical analysis of Layouts, for that a good reference is "A note on the algebra of CuTe Layouts"

GPUs don't know about tensors and their structures, operations are only performed on linear memory. Tensor structures need to be maintained by the kernels themselves. e.g. for a row-major matrix the matrix indices `i`, `j` will become index `i*C + j` where `C` is the number of columns. These kinds of mappings are pivotal in building GPU kernels since the shapes and structures of tensors can get very complicated.

A Layout we can easily define as nothing but a combination of the shape and stride (num of jumps to go from one element to next in a dimension) of a Tensor. It defines a mapping from the Tensor coordinate space to a flat array layout indices. Here stride and shape are tuples of matching dimensions. Calling the shapes S and the strides D we say

$$L = S : D$$

For example lets start with matrices, a row major $n \times m$ matrix will be represented as $(n, m) : (m, 1)$. Here we can see that increasing the first dimension (row) coordinate by 1 will increase the flat index by m and the second dimension (column) only increases it by 1. For getting the flat index/offset given i, j, we know we can get it by $i \times m + j = i \times m + j \times 1$. Or more generally we can see from how stride is defined that the flat index offset given indices $i = \{i_0, i_1, ..., i_d\}$ and stride $s = \{s_0, s_1, ..., s_d\}$,

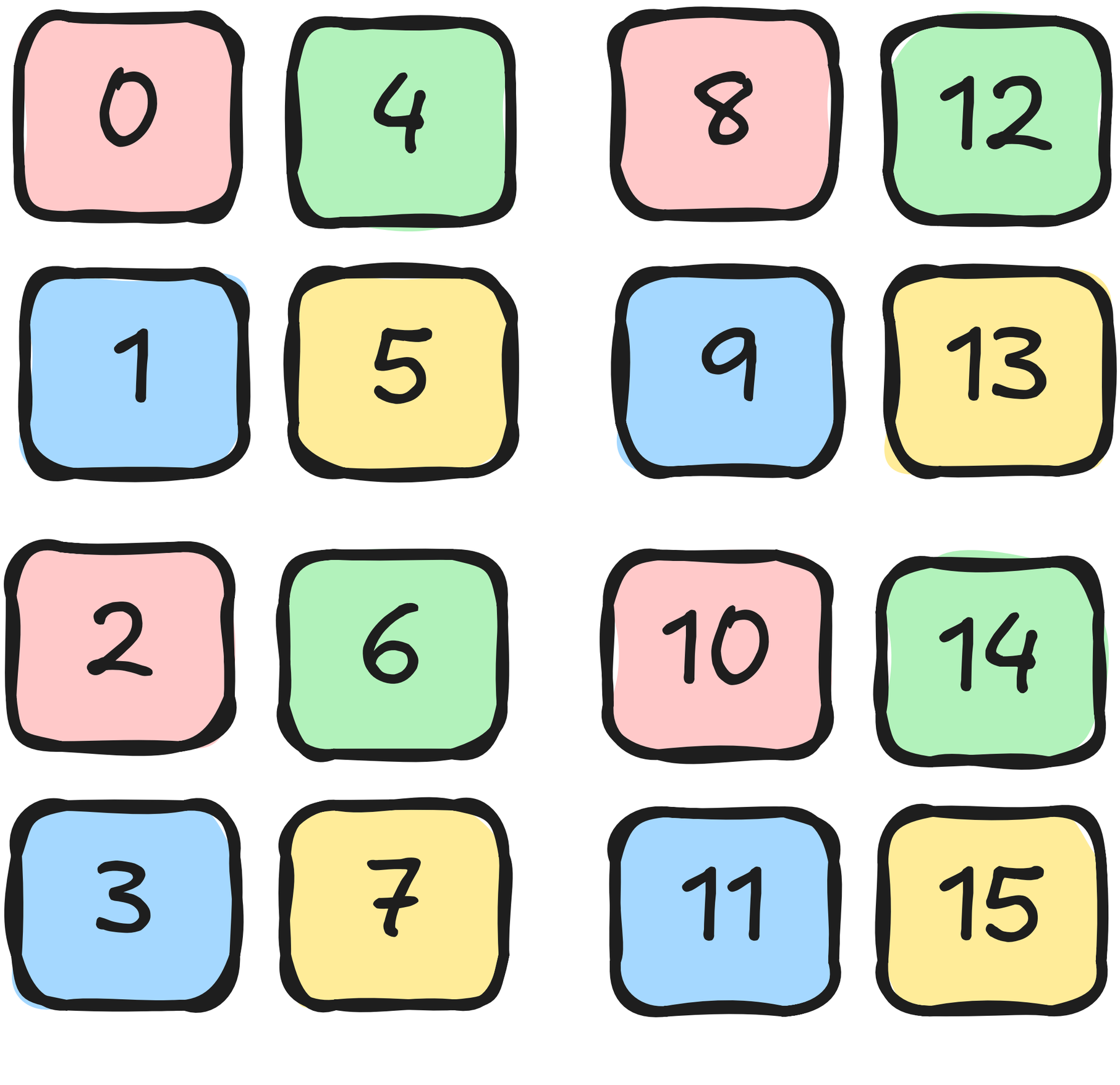
$$I = \sum i_k s_k = i \cdot s$$

Similarly the column-major matrix represented as $(n, m) : (1, m)$. Layouts can themselves be again made out of other Layouts. Lets analyze an example; $((2, 2), (2, 2)) : ((1, 4), (2, 8))$. Okay! Lets try some coordinates.

Coordinate $((i_0, i_1), (j_0, j_1))$	Calculation	Physical Offset
$((0, 0), (0, 0))$	$0(1) + 0(4) + 0(2) + 0(8)$	0
$((0, 1), (0, 0))$	$0(1) + 1(4) + 0(2) + 0(8)$	4
$((1, 0), (0, 0))$	$1(1) + 0(4) + 0(2) + 0(8)$	1

Coordinate $((i_0, i_1), (j_0, j_1))$	Calculation	Physical Offset
$((1,1),(0,0))$	$1(1) + 1(4) + 0(2) + 0(8)$	5
$((0,0),(1,0))$	$0(1) + 0(4) + 1(2) + 0(8)$	2
$((0,1),(1,0))$	$0(1) + 1(4) + 1(2) + 0(8)$	6
$((1,0),(1,0))$	$1(1) + 0(4) + 1(2) + 0(8)$	3
$((1,1),(1,0))$	$1(1) + 1(4) + 1(2) + 0(8)$	7

As we can see this is an interleaved layout (or swizzled). These kinds of layouts are generally used in GPU kernels to prevent memory bank conflicts.



Visualization of the swizzled layout

This is okay, but it is inconvenient to do offset calculations of a particularly complex layout. Another way of visualizing this is to think about the groups individually first $(2, 2)$ has a stride of $(1, 4)$ creating a $0 \to 4 \to 1 \to 5$ and this is repeated again in a $(2, 2)$ shape with each block in a $0 \to 2 \to 1 \to 3$ layout. This can be seen by factorizing the expanded offset.

$$\text{offset} = i_0 \cdot 1 + i_1 \cdot 4 + j_0 \cdot 2 + j_1 \cdot 8$$

$$\text{offset} = \underbrace{(i_0 + 2 \cdot j_0)}_{r} \cdot 1 + \underbrace{(i_1 + 2 \cdot j_1)}_{c} \cdot 4$$

This reduces a 4D layout into a 2D layout $(4, 4) : (1, 4)$ which is easy to visualize. But this is not possible for all layouts.

Layouts can also be operated on. This is the algebra part of the layout algebra. So we can have functions that can map from one layout to another layout. Such a function is equivalent to mapping of one flat index space to another. Hence

$$f : L_1 \to L_2 \implies f(i) : I(L_1) \to I(L_2)$$

Lets go through a few of these operations.

Coalescing

Coalescing is to simplify a layout. More formally, we can define *Rank* of a layout as the number of modes it has, coalesce operation reduces the number of modes i.e. to reduce its Rank. For example the layout $(2, 4) : (1, 2)$ is the same as $8 : 1$ for 1-D coordinates. So how do we get a coalesced layout? Say we have a layout $L = (s_0, s_1, ...) : (d_0, d_1, ...)$, (no nesting) how do we reduce a pair of layouts into one layout? We can check if the two modes are contiguous. When are they contiguous? When the next layout has a stride equaling the total flat indices of the former layout. This way they line up perfectly. so for $s_0 : d_0$ and $s_1 : d_1$ can reduce iff.

$$d_1 = s_0 \times d_0$$

and the two modes merge into $s_0 \times s_1 : d_0$. We can also see that 1 shape is an identity in reduction. i.e. $1 : d'$ and $s : d$ reduce to $s : d$. This is useful when we need to simplify some particularly gnarly tensor layouts. Coalescing can also be done by-mode i.e. we can only reduce some ranks e.g. from $4 \to 2$.

Composition

Composition chains two layouts together. Given a layout A and a layout B, the composition $B \circ A$ creates a new layout that first maps coordinates through A and then uses those resulting indices as coordinates into B. In other words, the output indices of A become the input coordinates of B.

$$(B \circ A)(i) = B(A(i))$$

Why is this useful? Composition is fundamental to the concept of tiling, where we partition a large data into smaller chunks that can be loaded into GPU's Shared Memory or further into Register Memory. We can do this with the composition operation as we'll see in later sections.

Lets work through an example. Take $A = (3, 2) : (1, 3)$ and $B = 6 : 2$. Layout A maps 2-D coordinates to 1-D indices in a 3×2 arrangement. Layout B maps a 1-D coordinate to offsets $\{0, 2, 4, 6, 8, 10\}$. The composition $B \circ A$ should give us a layout that maps 2-D coordinates directly to the physical offsets.

Coordinate (i, j)	$A(i, j)$	$B(A(i, j))$
$(0, 0)$	0	0
$(1, 0)$	1	2
$(2, 0)$	2	4
$(0, 1)$	3	6
$(1, 1)$	4	8
$(2, 1)$	5	10

We can verify this is the layout $(3, 2) : (2, 6)$. Now lets build up the general composition rules.

Single-mode B

In the simple case where $B = s_B : d_B$ is a single mode, composition just scales the strides of A by d_B:

$$(s_B : d_B) \circ (S_A : D_A) = S_A : (d_B \cdot D_A)$$

This works because B is linear — $B(k) = k \cdot d_B$ — so $B(A(i)) = d_B \cdot A(i)$. This is the case we saw in the example above: $(6 : 2) \circ ((3, 2) : (1, 3)) = (3, 2) : (2, 6)$.

Single-mode A

When B has multiple modes, the output of A needs to be decomposed into coordinates for B. Given $B = (s_0, s_1, ...) : (d_0, d_1, ...)$ and $A = s : d$, the flat indices from A (which are $\{0, d, 2d, ..., (s-1)d\}$) are split across B's modes using its shapes. This can only be done if either $d \mid s_0$ or $s_0 \mid d$. Without this, A's indices don't cleanly align with B's mode boundaries and composition is undefined.

There are two cases:

Case 1: $s_0 \mid d$ (stride skips past first mode). Since d is a multiple of s_0, every index $k \cdot d$ has $k \cdot d \mod s_0 = 0$, so B's first mode coordinate is always zero. We skip it entirely and recurse with reduced stride:

$$(s_0, s_1, ...) : (d_0, d_1, ...) \circ (s : d) = (s_1, ...) : (d_1, ...) \circ (s : d/s_0)$$

Case 2: $d \mid s_0$ (stride fits within first mode). Let $q = s_0/d$. The first q indices from A cycle through B's first mode before overflowing into the next.

- If $s \leq q$: all indices fit in the first mode. Result: $s : (d_0 \cdot d)$
- If $s > q$ and $q \mid s$: the first mode fills completely, and the rest recurse: $B \circ (s : d) = (q : d_0 \cdot d),\ ((s_1, ...) : (d_1, ...) \circ (s/q : 1))$ [1]

Lets trace through a concrete example. Take $B = (4, 3) : (1, 8)$ and $A = 6 : 2$.

We have $d = 2$ and $s_0 = 4$. Since $d \mid s_0$, we're in Case 2 with $q = 4/2 = 2$. Since $s = 6 > q = 2$ and $q \mid s$:

$$(4, 3) : (1, 8) \circ (6 : 2) = (2 : 1 \cdot 2),\ ((3,) : (8,) \circ (3 : 1))$$

For the recursive step, $d = 1$ and $s_0 = 3$, so $q = 3$ and $s = 3 \leq q$:

$$(3,) : (8,) \circ (3 : 1) = 3 : 8$$

Combining: $B \circ A = (2, 3) : (2, 8)$.

General composition: left-associative reduction

For the fully general case where both A and B are multi-mode, we reduce it to the cases above by processing A's modes left to right. Given $A = (s_0^A, s_1^A, ...) : (d_0^A, d_1^A, ...)$, we compose each mode of A one at a time (due to [1]):

$$B \circ A = (B \circ (s_0^A : d_0^A)),\ (B_{\text{rest}} \circ (s_1^A : d_1^A)),\ ...$$

Each single-mode composition consumes some of B's leading modes (via the cases above), and the unconsumed remainder B_{rest} carries forward for the next mode of A.

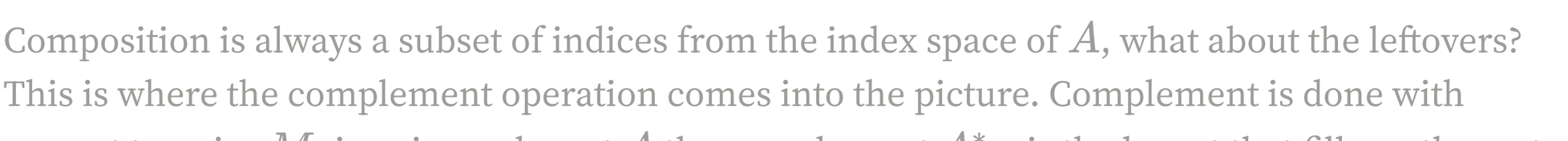
Composition of a Layout with a smaller Tile layout gives us a single first Tile from the original layout, we need some operation that can also do this for the rest of the layout.

Complement

Composition is always a subset of indices from the index space of A, what about the leftovers? This is where the complement operation comes into the picture. Complement is done with respect to a size M, i.e. given a layout A the complement A^*_M, is the layout that fills up the rest of M indices not covered by A. Complement only makes sense if the gaps left in M indices is shaped for filling up the space perfectly with a layout. For a layout $A = (s_0, s_1, ..., s_n) : (d_0, d_1, ..., d_n)$, this means $s_i \cdot d_i | d_{i+1}$ and $s_n \cdot d_n | M$. This means that the inner strides should fit into outer strides. We can calculate the complement of A as.

$$A^*_M = (d_0, \tfrac{d_1}{s_0 \cdot d_0}, \tfrac{d_2}{s_1 \cdot d_1}, ..., \tfrac{M}{s_n \cdot d_n}) : (1, s_0 \cdot d_0, s_1 \cdot d_1, ..., s_n \cdot d_n)$$

Here is an example for complement. Note we can reduce the $s_0 : d_0 = 1 : 1$ from this example. The gray part is the original layout and colored indices



Visualization of complement of a layout (from NVIDIA)

Division

Division operation splits a layout B into equal-sized tiles defined by layout A. As we have seen previously we can compose a layout and a tiler to get the first tile but to get all of them we need to also compose the complement with respect to a size M to get the rest of them.

$$B \oslash A := B \circ (A, A^*_M)$$

This splits each mode of B into two: the first mode is *within* tile from the composition and the second mode is indexing *across* tile from the complement. For a mode of B with shape s_B and stride d_B, and tiler shape t, we get

- Within-tile: shape t, stride d_B
- Across-tiles: shape s_B/t and stride $t \cdot d_B$.

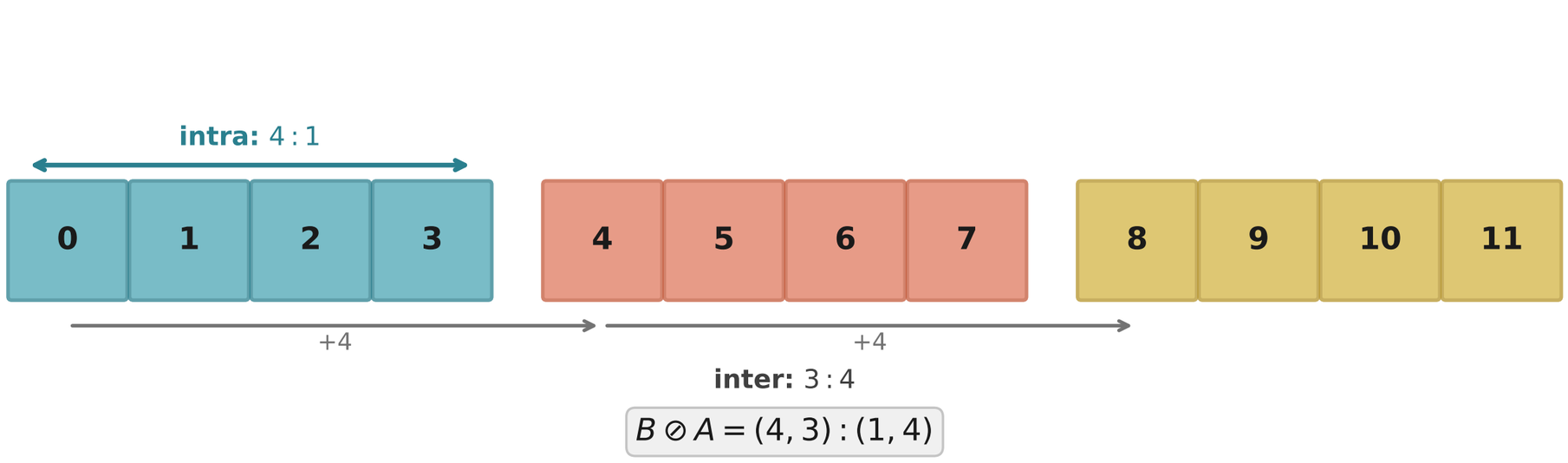
The original mode becomes a pair: $(t, s_B/t) : (d_B, t \cdot d_B)$. This operation doubles the *Rank* of the layout. Lets work through a few examples, first a simple 1D case. $B = 12 : 1$ and $A = 4 : 1$, A should divide B into 3 pieces. Hence;

Complement: $A^*_{12} = 3 : 4$ (the 3 tile offsets: $0, 4, 8$).

Concatenating: $(A, A^*) = (4, 3) : (1, 4)$.

Composing: $B \circ (4, 3) : (1, 4) = (4, 3) : (1, 4)$.

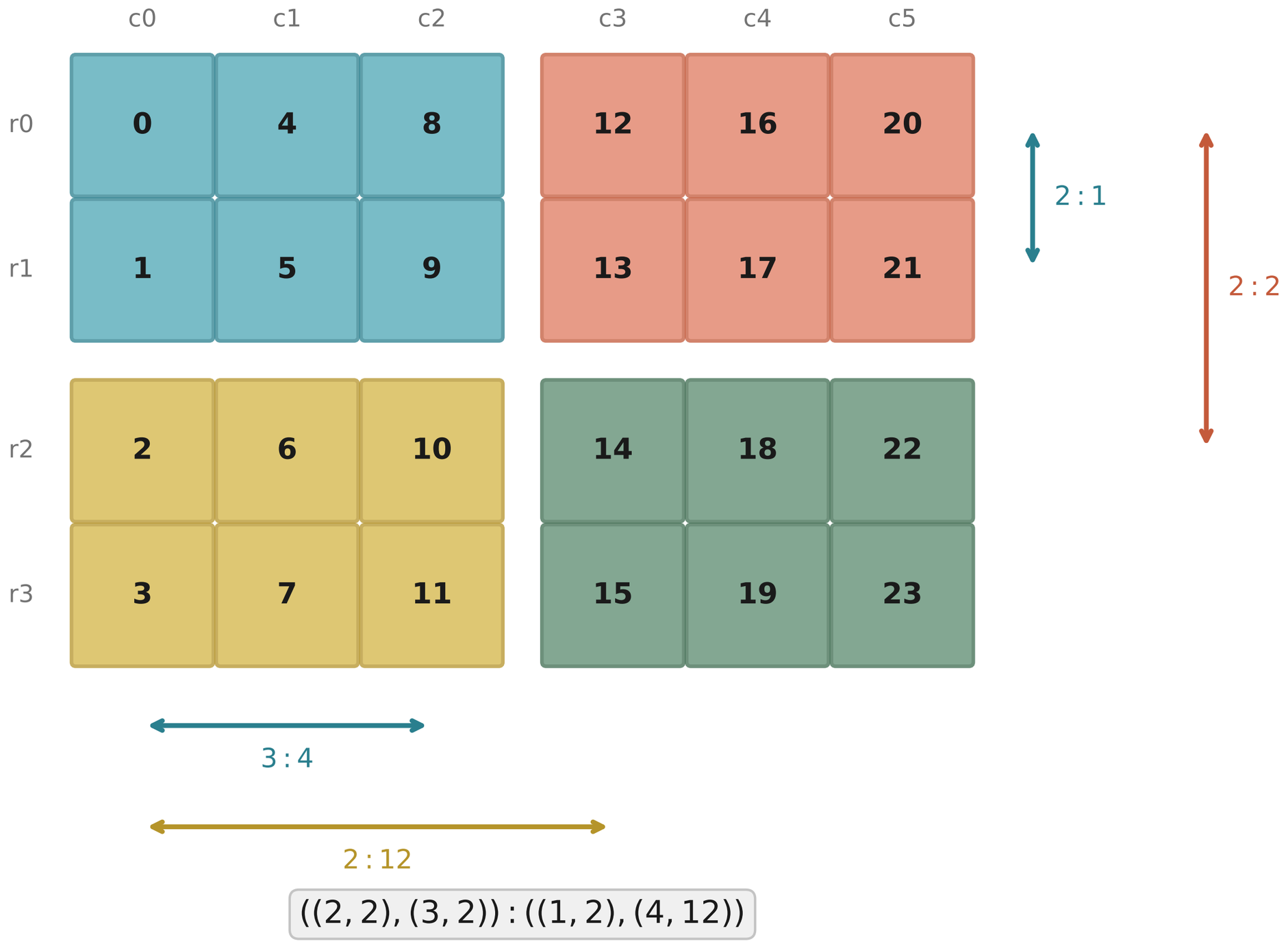
The first mode (size 4, stride 1) = within-tile indices $0, 1, 2, 3$. Second mode (size 3, stride 4) = tile offsets $0, 4, 8$. Three tiles of four elements each, covering all 12.



1D Example

Lets also work through a 2D example $B = (4, 6) : (1, 4)$ (4x6 column-major matrix), tiler $A = (2, 3)$ (2x3 tiles). Mode-by-mode:

- Mode 0: shape 4, tiler 2 $\rightarrow (2, 2) : (1, 2)$ — 2 rows in-tile, 2 tile-rows
- Mode 1: shape 6, tiler 3 $\rightarrow (3, 2) : (4, 12)$ — 3 cols in-tile, 2 tile-cols Logical divide result: $((2, 2), (3, 2)) : ((1, 2), (4, 12))$



2D Example

This is rank-4. Each original mode became a nested pair of (intra, inter).

This layout is difficult to work with due to its nesting, we can reduce it to easier layouts

Zipped, Tiled and Flat Divides

These are just convenience representations of the logical divide operation. Lets say a layout L has shape $(m, n, l, ...)$ and a tiler T with shape (t_m, t_n). The convenience forms are as below

- Logical: $((t_m, r_m), (t_n, r_n), ...)$
- Zipped: $((t_m, t_n), (r_m, r_n, l, ...))$
- Tiled: $((t_m, t_n), r_m, r_n, l, ...)$
- Flat: $(t_m, t_n, r_m, r_n, l, ...)$

Tiled Divide is especially interesting, it regroups the modes by role: all intra-tile modes together, all inter-tile modes together. The result is a clean two-level hierarchy — first group is “what’s inside a tile,” second group is “which tile.” This is the workhorse form that local_tile and partition_* use under the hood. It’s what you want when different parts of the kernel need to reason about tiles independently — the CTA picks its tile via the inter-tile mode, then threads work within it via the intra-tile mode.

Product

Where division breaks a layout into tiles, product does the opposite — it *replicates* a layout to fill a larger space. Given a layout A (the atom to replicate) and a layout B (the replication pattern), the product is

$$A \otimes B := (A, A^* \circ B)$$

The result is a two-mode layout: mode 0 is the atom A itself and mode 1 is $A^* \circ B$ which describes how the copies of A are arranged. The complement A^* finds the gaps between elements of A, and composing with B maps the replication pattern into those gaps.

Lets work through a 1D example. Take $A = 4 : 1$ (a contiguous atom of 4 elements) and $B = 3 : 1$ (replicate 3 times).

First we need the complement of A with respect to $M = \text{size}(A) \times \text{size}(B) = 12$. We get $A^*_{12} = 3 : 4$ – the three offsets $\{0, 4, 8\}$ where copies begin.

$$A^* \circ B = (3 : 4) \circ (3 : 1) = 3 : 4$$

So the product is $A \otimes B = (4, 3) : (1, 4)$. Notice anything? This is the same layout we got from dividing $12 : 1$ by $4 : 1$. That's the duality — division splits a layout into tiles, product builds one up from tiles. Two sides of the same coin.

Where product really shines is building *thread-value* layouts for distributing work across GPU threads. Say we have 4 threads each handling 2 values. The thread layout $T = 4 : 1$ and value layout $V = 2 : 1$, the product $T \otimes V$ tells us which elements each thread owns.

$T^*_8 = 2 : 4$, so $T^* \circ V = (2 : 4) \circ (2 : 1) = 2 : 4$. The product is $(4, 2) : (1, 4)$.

Thread 0 handles elements $\{0, 4\}$, thread 1 handles $\{1, 5\}$ and so on — each thread's values are spaced apart by the number of threads. This cyclic distribution is exactly what CuTe's `make_tiled_copy_tv` builds internally from a thread layout and value layout.

Blocked and Raked Products

For multi-dimensional layouts, the logical product works mode-by-mode just like division. Given atom A with shape (a_m, a_n) and replication pattern B with shape (b_m, b_n), the result has structure $((a_m, r_m), (a_n, r_n))$ where r denotes the replica modes. Blocked and raked products are two ways of reassociating these modes.

Blocked product groups like-modes with the atom inside: $((a_m, r_m), (a_n, r_n))$. Each atom occupies a contiguous block and replicas tile these blocks across the space. Think of it as "each thread gets a contiguous rectangle of elements."

Raked product reverses the nesting: $((r_m, a_m), (r_n, a_n))$. The replicas interleave within the atom dimensions, creating a cyclic distribution. Thread 0 gets element 0, thread 1 gets element 1, wrapping around. This is the classic GPU pattern for coalesced memory access — adjacent threads access adjacent memory locations.

Zipped and Tiled Products

Same convenience regrouping as with division. Given a layout L with shape $(m, n, l, ...)$ and tiler T with shape (t_m, t_n):

- Logical: $((m, t_m), (n, t_n), l, ...)$
- Zipped: $((m, n), (t_m, t_n, l, ...))$
- Tiled: $((m, n), t_m, t_n, l, ...)$
- Flat: $(m, n, t_m, t_n, l, ...)$

Putting it all together

Now lets see how all these layout operations come together in a real tiled GEMM kernel using CuTe's Python DSL. We compute $C = A \times B$ where A is $M \times K$, B is $K \times N$ (stored as $N \times K$, i.e. row-major transposed), and C is $M \times N$. The kernel tiles across all three dimensions with block tile sizes (b_M, b_N, b_K).

Setup and Tiled Copies

First we define shared memory layouts and tiled copy operations for moving data from global memory (GMEM) to shared memory (SMEM):

```
# Shared memory layouts - column-major for vectorized MMA access
sA_layout = cute.make_layout((b_m, b_k))  # (b_m, b_k) : (1, b_m)
sB_layout = cute.make_layout((b_n, b_k))  # (b_n, b_k) : (1, b_n)

# Thread layout for copy A: threads distributed across k-major order
tA = cute.make_layout(
    (num_threads // b_k, b_k), stride=(b_k, 1)
)
copy_atom_A = cute.make_copy_atom(
    cute.nvgpu.cpasync.CopyG2SOp(), dtype,
    num_bits_per_copy=dtype.width,
)
tiled_copy_A = cute.make_tiled_copy_tv(
    copy_atom_A, thr_layout=tA, val_layout=cute.make_layout((1, 1))
)

# Thread layout for copy B: with vectorized loads along n-major
num_vectorized = 4  # elements per vectorized load
copy_atom_B = cute.make_copy_atom(
    cute.nvgpu.cpasync.CopyG2SOp(), dtype,
    num_bits_per_copy=dtype.width * num_vectorized,
)
major_mode_size = b_n // num_vectorized
tB = cute.make_layout(
    (major_mode_size, num_threads // major_mode_size),
    stride=(1, major_mode_size),
)
tiled_copy_B = cute.make_tiled_copy_tv(
    copy_atom_B, thr_layout=tB,
    val_layout=cute.make_layout((num_vectorized, 1))
)
```

The `make_tiled_copy_tv` function takes a copy atom (the hardware copy instruction), a thread layout (how threads are mapped to tile elements), and a value layout (how many elements each

thread copies per invocation). This is where layout composition shines — CuTe composes these layouts to determine exactly which elements each thread is responsible for copying.

MMA Setup

The tiled MMA is set up similarly. We distribute threads in a $(T/16, 16, 1)$ layout across the M, N, K modes:

```
atoms_layout_mnk = cute.make_layout(
    (num_threads // 16, 16, 1), stride=(16, 1, 0)
)
tiled_mma = cute.make_tiled_mma(
    cute.nvgpu.MmaUniversalOp(abacc_dtype=acc_dtype),
    atom_layout_mnk=atoms_layout_mnk,
)
```

The Kernel

Inside the kernel, we first use **`local_tile`** to carve out each thread block's portion of the global tensors. The **`proj`** argument selects which modes the block tiler applies to:

```
bidx, bidy, _ = cute.arch.block_idx()
cta_coords = (bidx, bidy, None)

# gA: (b_m, b_k, k_tiles), gB: (b_n, b_k, k_tiles), gC: (b_m, b_n)
gA = cute.local_tile(mA, block_tiler, cta_coords, proj=(1, None, 1))
gB = cute.local_tile(mB, block_tiler, cta_coords, proj=(None, 1, 1))
gC = cute.local_tile(mC, block_tiler, cta_coords, proj=(1, 1, None))
```

Then we partition the tiles across threads for both copying and computation:

```
tidx, _, _ = cute.arch.thread_idx()
thr_mma = tiled_mma.get_slice(tidx)

# Partition copy source (GMEM) and destination (SMEM) for each thread
thr_copy_A = tiled_copy_A.get_slice(tidx)
tAgA = thr_copy_A.partition_S(gA)  # (cpy, cpy_m, cpy_k, k_tiles)
tAsA = thr_copy_A.partition_D(sA)  # (cpy, cpy_m, cpy_k)

thr_copy_B = tiled_copy_B.get_slice(tidx)
tBgB = thr_copy_B.partition_S(gB)  # (cpy, cpy_n, cpy_k, k_tiles)
tBsB = thr_copy_B.partition_D(sB)  # (cpy, cpy_n, cpy_k)

# Partition SMEM and output for MMA
tCsA = thr_mma.partition_A(sA)
tCsB = thr_mma.partition_B(sB)
tCgC = thr_mma.partition_C(gC)
tCrC = tiled_mma.make_fragment_C(tCgC)
tCrC.fill(0.0)
```

Each **`partition_*`** call internally uses layout composition and division to split the tile into per-thread pieces. The **`cpy`** mode in the copy partitions captures both the vectorization width and the number of copy operations each thread performs.

The Main Loop

The k-tile loop copies each (b_M, b_K) and (b_N, b_K) tile from GMEM to SMEM using async copies, then performs the MMA:

```
k_tiles = cute.size(tAgA, mode=[3])

for k in range(k_tiles):
    cute.copy(tiled_copy_A, tAgA[None, None, None, k], tAsA, pred=tApA)
    cute.copy(tiled_copy_B, tBgB[None, None, None, k], tBsB, pred=tBpB)

    cute.arch.cp_async_commit_group()
    cute.arch.cp_async_wait_group(0)
    cute.arch.sync_threads()

    cute.gemm(tiled_mma, tCrC, tCsA, tCsB, tCrC)
    cute.arch.sync_threads()
```

The **`pred`** arguments handle boundary conditions — when M, N, or K aren't exact multiples of the tile sizes, predicate tensors mask out-of-bounds accesses. These predicates are themselves built using layout operations on identity tensors.

Epilogue

Finally the accumulated results are written back to global memory, again with predication for bounds checking:

```
cC = cute.make_identity_tensor(gC.shape)
tCpC = thr_mma.partition_C(cC)
predC = cute.make_rmem_tensor(tCrC.layout, cutlass.Boolean)
residue_m = mC.shape[0] - b_m * bidx
residue_n = mC.shape[1] - b_n * bidy
for i in range(cute.size(tCrC.shape)):
    predC[i] = cute.elem_less(tCpC[i], (residue_m, residue_n))
atom = cute.make_copy_atom(cute.nvgpu.CopyUniversalOp(), mC.element_type)
cute.copy(atom, tCrC, tCgC, pred=predC)
```

The full kernel is available in the companion repo.

Notice how the layout algebra permeates the entire kernel — from defining how threads map to data (**`make_tiled_copy_tv`**), to carving tiles (**`local_tile`**), to splitting work across threads (**`partition_*`**), to bounds checking (**`make_identity_tensor`** + **`elem_less`**). Without these abstractions, we'd be manually computing thread-to-element mappings with error-prone index arithmetic. The layout algebra replaces all of that with composable, type-safe operations.

Closing thoughts

CuTe's layout algebra turns what would be pages of error-prone index arithmetic into a handful of composable operations — composition, complement, and division — that are easier to reason about. But I want to be honest about the tradeoff here.

GPU memory layouts are inherently complex. Swizzled shared memory, bank conflict avoidance, mixed-radix thread-to-data mappings — this complexity is *intrinsic* to the hardware. CuTe doesn't eliminate it; it repackages it into a different formalism. You're trading one kind of complexity (raw index math) for another (an algebra with its own rules, edge cases, and debugging challenges). As one Reddit commenter put it, CUTLASS is *"a typical example of a library with so much abstractions that makes complicated things simple and simple things complicated."*

The learning curve is steep. You need to internalize layouts, composition, complement, division, cosize, cotarget, tiled copies, MMA atoms — a whole vocabulary before you can write your first kernel. For someone who already understands raw CUDA well, the value proposition is debatable: you're investing significant effort to learn abstractions that, at the end of the day, generate the same PTX. The payoff comes when you need to support multiple GPU architectures, swap out MMA instructions, or restructure tiling strategies without rewriting everything — that's where the composability genuinely shines. But for a one-off kernel on a single architecture, you might be better off with raw CUDA and a good understanding of your hardware.

If you want to dig deeper, the CuTe documentation is thorough, and the CUTLASS Python DSL makes it easier to experiment with these ideas interactively — the Python interface significantly reduces the compile-time pain that plagues the C++ templates.

Footnotes

1. $(s_0, s_1, \ldots) : (d_0, d_1, \ldots)$ and listing them elementwise $(s_0 : d_0), (s_1 : d_1), \ldots$ is the same. ↩ ↩²

Share

Read Next

Twelve Attempts at an FP4 Kernel

A worklog of NVFP4 kernels, failed experiments, and one stubborn memory bus

February 28, 2026

Hierarchical Navigable Small Worlds

A deep dive into HNSW, an approximate nearest neighbor search algorithm.

February 8, 2026

© 2026 amandeep singh. All rights reserved.